COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE 21S3
DATE PERFORMED:	10/06/2026	**DATE SUBMITTED:**	10/06/2026
NAME:	Busa, Emmanuel A. Geronimo, Joncel Zairho G.	**INSTRUCTOR:**	Engr. Jimlord Quejado

HOA 11.1 Basic Algorithm Analysis

1. PROCEDURE OUTPUT

ILO A: Measure the runtime of algorithms using theoretical analysis

Assess if two arrays have any common values.

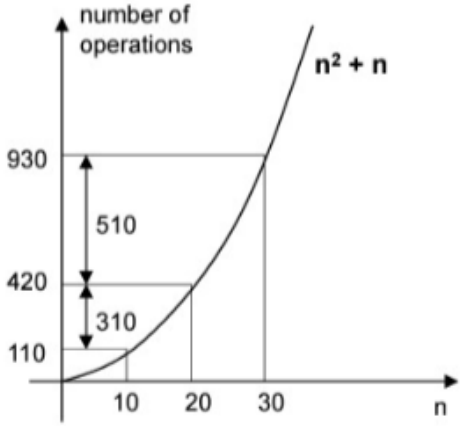
```
bool diff(int *x, int *y){
    for(int i = 0; i < y.length; i++){
        if(search(x, y[i])!= - 1){
           return false;
        }
    }
    return true;
}
```

Performing worst case analysis:

- Let m = the length of array x.
- Let n = the length of array y.
- The loop in diff repeats n times.
- Each call to search requires m comparisons.

Given the above information, the total number of comparisons in the worst case is: **m x n**

Assume that m = n. (The arrays are the same size)
Provide an analysis of the graph for the worst case of the algorithm.



n (size of array)	number of operations
10	110
20	420
30	930

Analysis:
The worst case number of comparisons is m × n. Since m = n, it becomes n^2. Including the loop operations, the graph shows the runtime as approximately n² + n, so the algorithm has quadratic time complexity, O(n^2). As the array size increases, the number of operations grows rapidly.

ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis

Step 0: Begin by including the following headers:

```
#include <iostream>
#include <vector>
#include <chrono>
#include <random>
#include <algorithm>
#include <numeric>
```

Step 1: Define the STUDENT class as follows:

```
class Student{
    private:
        std::pair<int, int> name;
        bool vaccinated;

    public:
        //constructor
        Student(std::pair<int, int> n, bool v) : name(n), vaccinated(v){
        }
        // Getters
        auto get_name() { return name; }
        auto is_vaccinated() { return vaccinated; }
```

Step 2: Overload the required operators for the Student class so that a vector of students can be sorted using the STL std::sort() function.

```
        // Two people are same if they have the same name
        bool operator ==(const Student& p) const {
            return this->name == p.name;
        }
        // The ordering of a set of people is defined by their name
        bool operator< (const Student& p) const {
            return this->name < p.name;
        }
        bool operator> (const Student& p) const {
            return this->name > p.name;
        }
};
```

Step 3: The following function lets us generate a student from random data:

```
auto generate_random_Student(int max) {
    std::random_device rd;
    std::mt19937 rand(rd());
    // the IDs of Student should be in range [1, max]
    std::uniform_int_distribution<std::mt19937::result_type> uniform_dist(1, max);
    // Generate random credentials
    auto random_name = std::make_pair(uniform_dist(rand), uniform_dist(rand));
    bool is_vaccinated = uniform_dist(rand) % 2 ? true : false;
    return Student(random_name, is_vaccinated);
}
```

For the next important function, we will implement the use of **std::chrono** for experimental analysis. The following code is used to run and test the output of our implementation.

```
void search_test(int size, Student p) {
    std::vector<Student> people;
    // Create a list of random people
    for (auto i = 0; i < size; i++)
        people.push_back(generate_random_Student(size));
    std::sort(people.begin(), people.end());
    // To measure the time taken, start the clock
        std::chrono::steady_clock::time_point begin =
    std::chrono::steady_clock::now();
        bool search_result = needs_vaccination(p, people);
    // Stop the clock
        std::chrono::steady_clock::time_point end = std::chrono::steady_clock::now();
        std::cout << "Time taken to search = " <<
            std::chrono::duration_cast<std::chrono::microseconds>
            (end - begin).count() << " microseconds" << std::endl;
        if (search_result)
            std::cout << "Student (" << p.get_name().first << " " << p.get_name().second
    << ") " << "needs vaccination." << std::endl;
        else
            std::cout << "Student (" << p.get_name().first << " " << p.get_name().second
    << ") " << "does not need vaccination." << std::endl;
}
```

5. The following function implements our logic for whether a vaccination is needed:

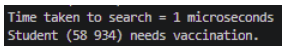
```
bool needs_vaccination(Student P, std::vector<Student>& people) {
    auto first = people.begin();
    auto last = people.end();
    while (true)
    {
        auto range_length = std::distance(first, last);
        auto mid_element_index = std::floor(range_length / 2);
        auto mid_element = *(first + mid_element_index);
        // Return true if the Student is found in the sequence and
        // he/she's not vaccinated
        if (mid_element == P && mid_element.is_vaccinated() == false)
            return true;
            else if (mid_element == P && mid_element.is_vaccinated() == true)
            return false;
        else if (mid_element > P)
            std::advance(last, -mid_element_index);
        if (mid_element < P)
            std::advance(first, mid_element_index);
        // Student not found in the sequence and therefore should be vaccinated
        if (range_length == 1)
            return true;
    }
}
```

6. Finally the driver code is implemented as follows:

```
int main() {
    // Generate a Student to search
    auto p = generate_random_Student(1000);
    search_test(1000, p);
    search_test(10000, p);
    search_test(100000, p);
    return 0;
}
```

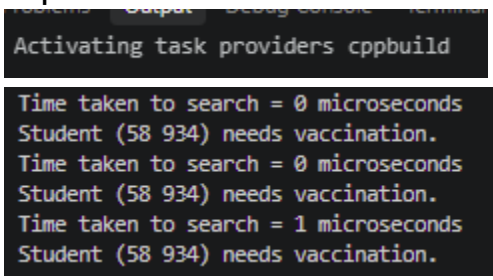
Fill out the table below and include in section 6 table 11-1.

Input Size (N)	Execution Speed	Screenshot	Observation(s)
1000	0 microsecond/s	Time taken to search = 0 microseconds Student (58 934) needs vaccination.	The search completes instantly
10000	0 microsecond/s	Time taken to search = 0 microseconds Student (58 934) needs vaccination.	Execution time is still 0 microsecond. Increasing the search collection by 10 times seems to have no difference in the performance

100000	1 microsecond/s		Minimal increase to 1 microsecond, even with the massive increase of 10 times the search collection, the search performs really well

Provide your own analysis of the outcome listed in this table.

Output:



2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO A: Measure the runtime of algorithms using theoretical analysis & ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis

Problem 1: Consider a program that returns true if the elements in an array are unique.
Algorithm Unique(A)

```
for (int i = 0; i < n - 1; i++) {
    for (int j = i + 1; j < n; j++) {
        if (A[i] == A[j]) {
            return false;
        }
    }
}

return true;
```

Zairho:
Theoretical Analysis**:**
The algorithm uses two nested loops to compare every pair of elements in the array

Experimental Analysis:
The runtime increased as the input size also increased.

Analysis and Comparison:
The experimental results agree with the theoretical analysis as when the input size was doubled, the runtime increased by about 4 times

Emmanuel:
Theoretical Analysis:
The algorithm uses two loops, so it makes more comparisons when the array gets bigger. The time complexity is $O(n^2)$

Experimental Analysis:
When the input size increases, the execution time also increases because more elements needs to be compared with each other

Analysis and Comparison:
The theoretical and experimental results are almost the same. Both shows that the algorithm gets slower when the input becomes bigger

Problem 2: Consider another program that raises a number x to an arbitrary nonnegative integer, n.

```
long long rpower(long long x, int n) {
    if (n == 0)
        return 1;
    else
        return x * rpower(x, n - 1);
}

//Binary Recursive Power
long long brpower(long long x, int n) {
    if (n == 0)
        return 1;

    if (n % 2 == 1) {
        long long y = brpower(x, (n - 1) / 2);
        return x * y * y;
    }

    if (n % 2 == 0) {
        long long y = brpower(x, n / 2);
        return y * y;
    }

    return 1;
}
```

Zairho:
Theoretical Analysis:
The brpower algorithm reduces n by about half in every call. Therefore it has a time complexity of O(log n) making it more efficient than rpower.

Experimental Analysis:
The runtime of brpower remained about 0.0000001 for all tested values of n from 10 to 50

Analysis and Comparison:
The experimental results show that brpower is very fast even as n increases

Emmanuel:
Theoretical Analysis:
The rpower algorithm has O(n) because it decreases n one by one. The brpower is faster with O(log n) because it divides n by 2 everytime.

Experimental Analysis:
The rpower takes more time for bigger inputs, while brpower is faster because it uses less recursive calls

Analysis and Comparison:
The results shows that brpower is more efficient than rpower, especially for bigger values of n

Conclusion:
Provide the following:

- Summary of lessons learned:

  Zairho: I learned how to measure, implement, and analyze algorithm performance using both std::chrono and the theoretical Big-O analysis. I observed that choosing the right algorithm structure like using Binary Search can mean the difference between an instant execution and a system crash

  Emmanuel: I noticed how Big-O helps us know how efficient an algorithm is and how it performs when the input gets bigger.

- Analysis of the procedure

  Zairho: The procedure helped me understand the difference between theoretical and experimental analysis.

  Emmanuel: The procedure made me understand how we can compare the theoretical and actual performance of an algorithm

- Analysis of the supplementary activity

  Zairho: The supplementary activity helped me understand the algorithm easier by allowing me to compare different approaches to solving a problem. Showing me that choosing an efficient algorithm is important.

  Emmanuel: I learned that choosing a better algorithm can make a big difference in the program performance.

Concluding statement / Feedback: How well did you think you did in this activity? What are your areas for improvement?

Zairho: I feel like I did alright in this activity because I was able to implement algorithms, measure their runtime using std::chrono and compare the experimental results with the theoretical

Emmanuel: I think I did well enough in this activity to learn about basic algorithm and answer the analysis part but I still need to improve more on Big-O and recursive algorithms

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used:

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use:

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.